

Software Engineering

System Architecture

System and Software Architecture

The **last responsible moment** is the strategy of delaying a decision until the moment when the cost of not making the decision is greater than the cost of making it.

Design decisions made for a software architecture can be among the **most important** for a software system and they can be among the most **difficult to change later**.

With traditional software architectures, decisions were made very early in the project. In order to design an **evolutionary architecture**, it is beneficial to delay a decision until the last responsible moment. This lessens the possibility that a **premature decision** will be made. Decisions that are made too early are very risky because they may end up being incorrect and then they will result in **costly rework**.

Source: [Software Architect's Handbook by Joseph Ingeno \(O'Reilly\)](#)

What is the difference between System and Software Architecture?

System architecture encompasses the overall design and organisation of a system, which includes both hardware and software components.

It defines the structure and behavior of the entire system, including the software, hardware, networks, data storage, and other critical elements.

Software architecture refers to the high-level structure of a software system and the discipline of creating such structures.

It defines how the software components or modules are organised and how they interact with each other.

Introduction to Patterns

- In software engineering a pattern is a reusable design to common problems within a context
- As a design it is more like an approach than a finished solution
 - Has to be applied to a specific problem to be useful (code-able)



Pattern

- Abstract
- Structure
- Template
- Flexible in application



Algorithm

- Concrete
- Finite set of actions
- Implementation
- Applicable to a specific problem

- A solution proposal should be applicable to multiple problems to be considered as a pattern

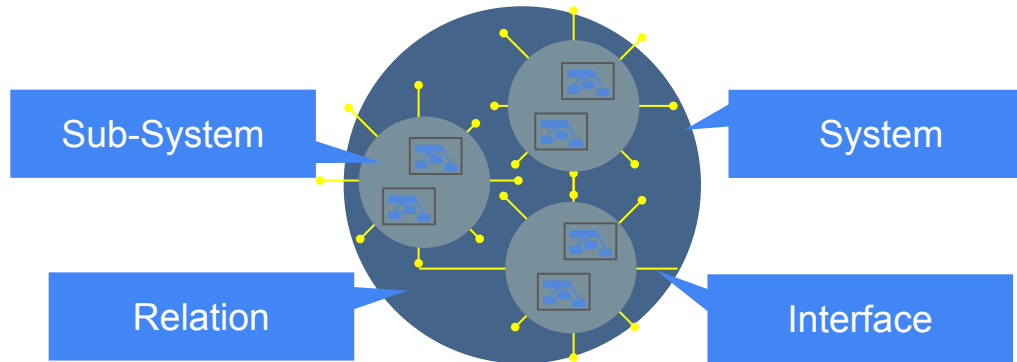
What is System Architecture?

Architectural Styles

- Software architecture strives to achieve
 - high cohesion by having **more relations between object than between sub-systems**
 - low coupling by **reducing knowledge about internal implementations** of one sub-system by another
- Software architecture includes specifications of
 - System composition
 - Control flow
 - Handling of conditions
 - Communication between sub-systems
- Architectural styles are patterns that are used as basis for software architectures
 - Model / View / Controller
 - Client / Server
 - Peer-to-Peer
 - Etc.

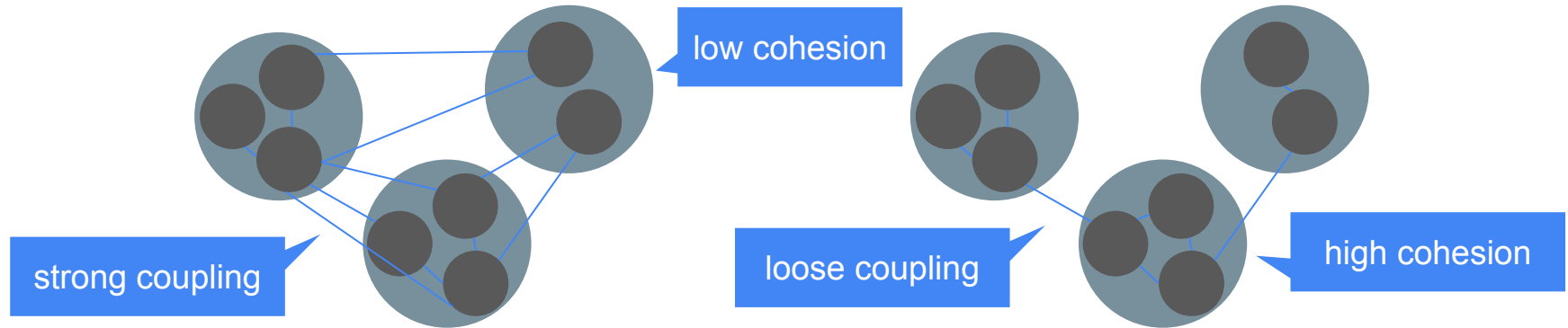
Decomposition – Sub-Systems and Services

- Decomposition is the activity of **identifying sub-systems, services** and their **relationships**
- **Sub-system**
Collection of classes, associations, functions and events that are related. We can create sub-systems out of the **objects** from **object model**.
- **Service**
Group of externally visible operations provided by a sub-system (interfaces). We create services out of **use cases** from the **functional model**.



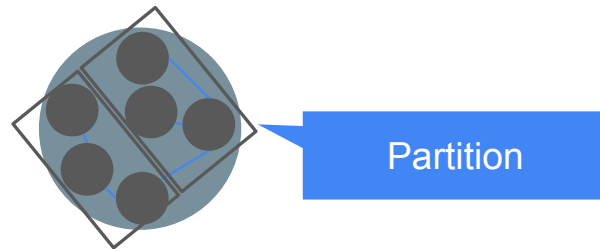
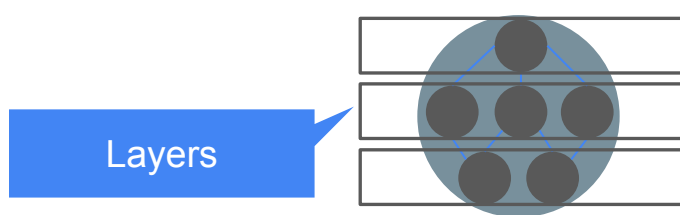
Decomposition - Goals

- Reduce complexity through **high cohesion** and **loose coupling**
- Cohesion is the strength of dependencies within a sub-system
High: sub-system contains **related** objects performing similar tasks
Low: sub-system contains **unrelated** objects
- Coupling is the strength of dependencies between sub-systems
Strong: changes to one sub-system **influence** another one heavily
Loose: sub-systems work **independently**, as long as the interface stays stable



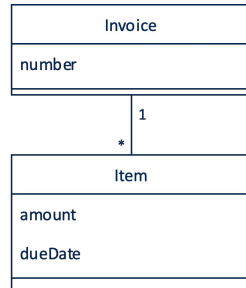
Decomposition – Layering & Partitioning

- Basic techniques in decomposition are layering or partitioning
- **Partitioning**
Divides a system into independent peer sub-systems
 - Sub-system provide services to each other on the same level of abstraction
 - Each sub-system is responsible for for a different class of service
- **Layering**
Decomposes sub-systems in a hierarchy
 - Layer is one or more sub-systems providing services to higher-level layers
 - Higher-level layers are dependent on lower ones while a lower-level layer doesn't know of higher-levels
 - Can be closed (layer can only access the one below) or open (layer can access all level below)



Data Management

- Deals with persistency of objects
- Main forms of persistency
 - Database => very common
 - File => uncommon
- Main considerations for persistency
 - Volume and throughput of data
 - Extensibility of data format
 - Archiving volume and frequency
 - Distribution of data
 - Data integration
- Datawarehousing
 - DWH is heavily dependant on data management and design!



- Object models can be mapped to relational databases
 - Class => Table
 - Member => Column
 - Instance => Row / Entry
 - One-to-many => foreign key
 - Many-to-many => Relation table with to keys
- Methods and virtual fields are not mapped

Invoice

key	number
1	2020_1
2	2020_2

Item

key	amount	due_date	invoice_fk
1	200	13.10.2020	1
2	199	10.12.2020	1

System Architecture Patterns

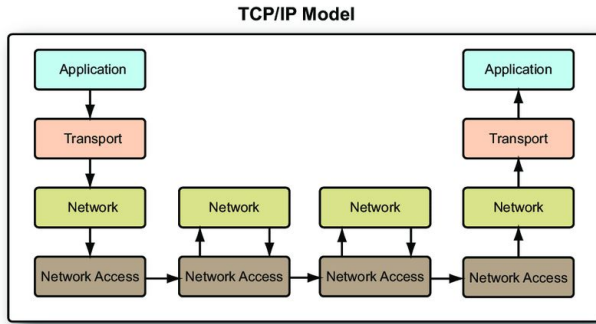
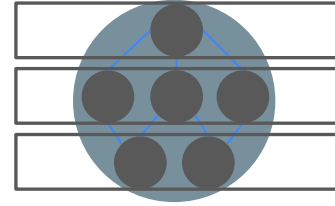
Architectural Patterns

- Architectural patterns **define the basic structure** of an application
- In architecture for buildings -> architects choose from several styles
 - They change details according to customer requirements
 - This enables reuse of engineering knowledge
- We apply the same approach in software architecture
- Elements of software architectures
 - **Components** – sub-systems
Computational units with programming interfaces like database, layer, object
 - **Connectors** – communication
Interactions and the protocols between components like methods, events, shared data
- We will take a look at a few architectural patterns
 - Layered, N-Tier, SOA, REST, Microservice, SCS, Event-Driven, ETL, MVC



Layered Architecture

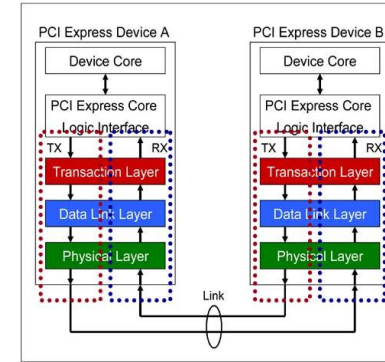
- Goal: decomposition (originally proposed by [Dijkstra in 1968](#))
- Famous examples for this are TCP/IP and PCI Express



Anantavasilp, Isara. (2010). Supervised Machine Learning Assisted Real-Time Flow Classification System.

Pros

- Reusability of layers
- Standardization
- Low coupling
- Testability



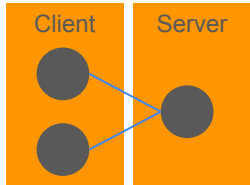
PCI Express Device Layers

Cons

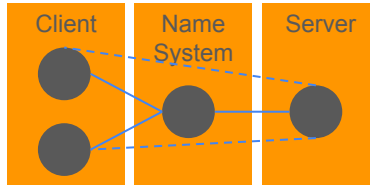
- Lower layer changes might require higher-level changes as well
- Lower efficiency than direct connection
- Higher effort in setup

N-Tier Architecture

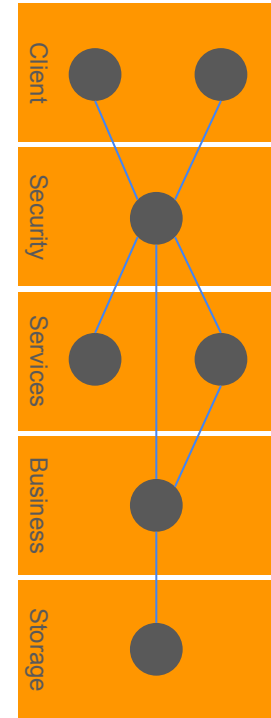
- Based on layer pattern
- Simplest form Client / Server (two tier)
- Most complex in n-tier (n layers)



Client /
Server



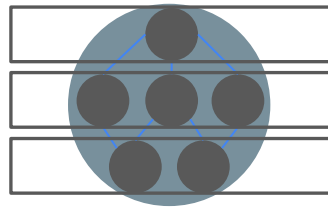
Three Tier



N Tier

rising complexity and looser coupling

Service Oriented Architecture (SOA)



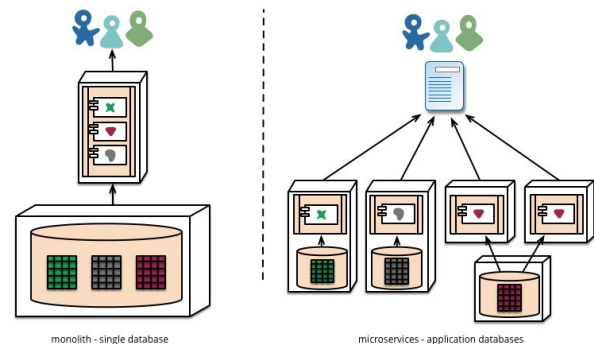
- Deals with the **communication** (between layers)
- **Services** have properties as they
 - represent a business activity
 - are self-contained
 - are black-boxed
 - can be composed of other services (in layers)
- Services are provided by components through communication **protocol**
- Protocols describe how services pass information through **metadata**
- SOA can be implemented in many ways
 - SOAP
 - Messaging
 - REST
 - gRPC, GraphQL

Representational State Transfer Architecture (REST)

- Defines **how** services should be build with a set of **constraints**
- **Stateless**: client context is not stored, every request contains all information needed
- Services that conform to constraints are **RESTful**
- Common protocol is HTTP
 - GET, POST, PUT, PATCH, DELETE
- Common payload format
 - JSON, XML, HTML
- Other constraints
 - **Separation of concerns** (separation of UI, data storage, business logic, security, etc)
 - **Cacheability** (responses can be cached to improve performance)
 - **Composability** (services can be composed of other services and are not aware of the layers)
 - **Uniformity**
 - identification of resources in request, manipulation through the representation, no hard coded client knowledge through hypermedia

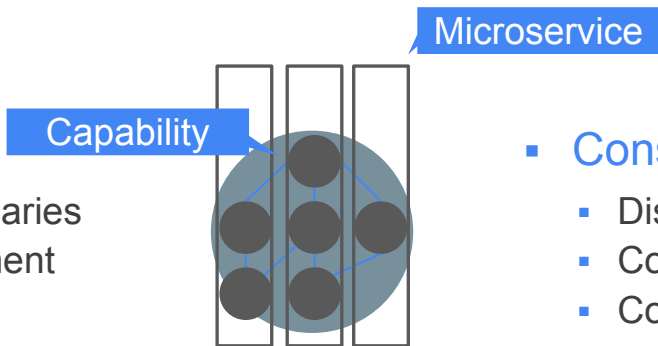
Microservice Architecture

- Specialized form of SOA for
 - Developing a **single application** as a set of **small services**
 - where each service is running in its **own process**,
 - using **lightweight communication** mechanisms (HTTP),
 - built around **business capabilities**,
 - **independently deployable** through automated processes,
 - with minimum centralized control (individual programming **languages** and **storage per service**).



■ Pros

- Strong module boundaries
- Independent deployment
- Technology diversity



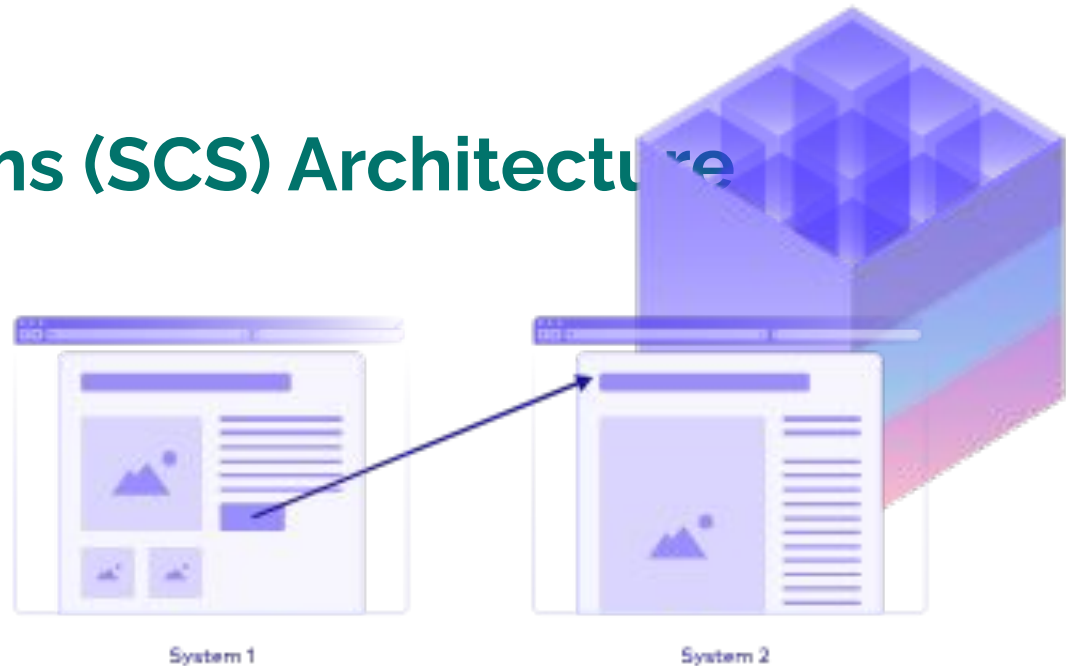
■ Cons

- Distribution
- Consistency
- Complexity

Self Contained Systems (SCS) Architecture

Each SCS is an **autonomous web application**. For the SCS's domain, all data, the logic to process that data and all code to render the web interface is contained within the SCS. An SCS can fulfill its primary use cases on its own, without having to rely on other systems being available.

Source: <https://scs-architecture.org/>



■ Pros

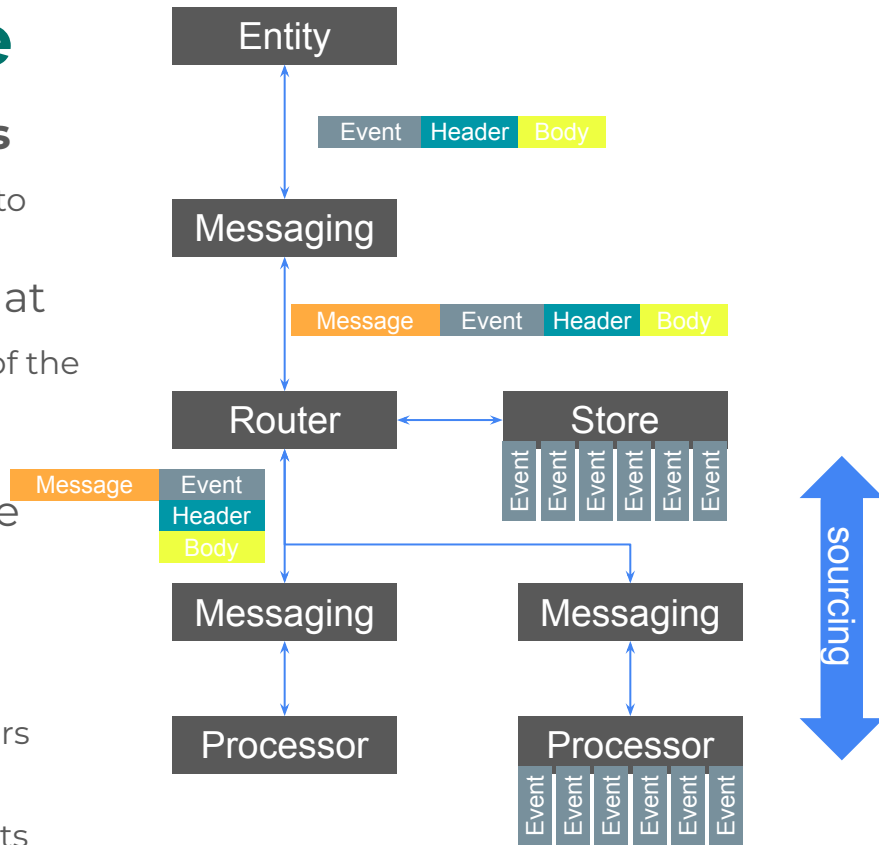
- Strong module boundaries
- Independent deployment
- Technology diversity
- Less overhead than microservices
- Can be developed in one team autonomously

■ Cons

- Distribution
- Consistency
- Complexity, but less complex than microservices

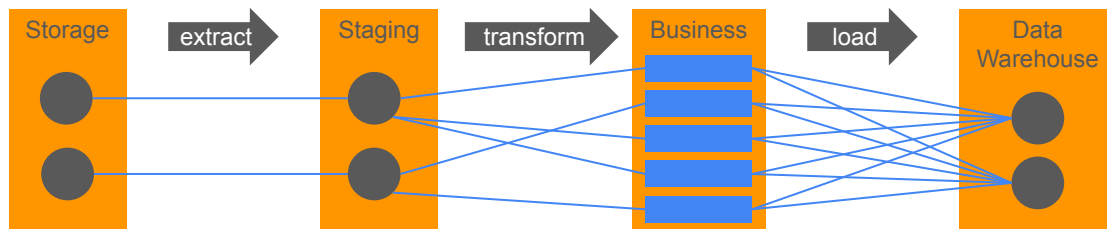
Event Driven Architecture

- An architecture centered around **events**
 - production, detection, consumption and reaction to events
- Event is a significant **change of state** that
 - contains a header (metadata) and a body (details of the change)
 - causes an event notification
- Elements of an event driven architecture
 - **entities** produce (emit) and consume events
 - **messaging systems** distribute event notifications
 - **routers** mediate events between systems
 - entities can **subscribe** to certain events with routers
 - **stores** persist events
 - **sourcing** enables recreation of state through events



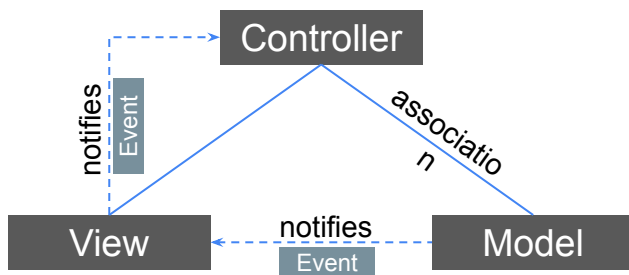
Extract / Transform / Load Architecture

- Specialized for data processing
- Three main parts (ETL)
 - Extraction of data from various databases
 - Transformation of that data through consolidation in business entities
 - Loading of business entities in data warehouse systems



Model / View / Controller Architecture

- Used in many user interface frameworks ([first appeared in 1979](#))
- Famous examples are Rails, ASP.NET, Angular, Backbone, Spring, JavaFX, etc.



- **Pros**
 - Decouples data from view
 - Well suited for user interfaces
 - Can be used for maintaining consistency
- **Cons**
 - Controller handles view specific data modifications
 - Other variants: Model View ViewModel (MVVM)

Summary: System Architecture Patterns

- Layered Architecture
- N-Tier Architecture
- Service Oriented Architecture (SOA)
- Representational State Transfer Architecture (REST)
- Microservice Architecture
- Self Contained Systems (SCS) Architecture
- Event Driven Architecture
- Extract / Transform / Load Architecture
- Model / View / Controller Architecture

COTS: Commercial-off-the-shelf - Make or Buy decisions

- Usage of different levels of ready-made software
 - **Libraries** – specific functionality (sorting algorithms, data structures, etc.)
 - NPM Packages, ZIP, Logging, HTTP, Unit Testing, GSON, JAXP, etc.
 - **Frameworks** – assortment of libraries tied to specific sub-systems (UI frameworks, messaging framework, etc.)
 - React, Angular, Spring, Fiori, Rails, Apache Camel, Kafka, etc.
 - **Platforms** – solutions (of frameworks, libraries and more) specific to a business case
 - SAP FI/CD/ERP/etc, MS Power BI/Dynamics/etc, Salesforce, Guidewire Suite, Google Suite, Amazon WorkDocs/Mail/Connect/etc,
- Depending on the usage the approach to mapping business requirements to solution design will change dramatically
 - Heavy reuse in platforms, fastest time to market, lowest risk => yet high invest and specialization – low code
 - Medium reuse in frameworks, higher flexibility and risk => yet higher development and design costs - code
 - Low reuse in libraries, highest flexibility => yet large development, architecture and design costs, highest risk



In large scale projects you will almost certainly see usage of platforms.
Probably mixed with frameworks and libraries.
Seldomly developments from scratch!

Exercise - 10 minutes

In groups of 3-5:

Collect pros and cons to use

Cloud (Servers in outsourced data centers) versus

On-Premise (Servers in your own data centers)

System Architecture Patterns

How does an Architectural Pattern affect the responsibility split between multiple software engineering teams working on the same project or product?

System Architecture Patterns

Conway's Law: [O]rganizations which design systems (in the broad sense used here) are constrained to produce designs which are copies of the communication structures of these organizations.

—Melvin E. Conway, computer programmer, 1967

Source: [Wikipedia: Conway's Law](#)

